

Лабораторна робота 8. OTA (over-the-air) оновлення програми на ESP32.

Завдання: Запрограмуйте **ESP32** програмою, котра повільно блимає вбудованим світлодіодом із частотою, заданою викладачем, а також містить код для **OTA** оновлення програми через **WiFi** по натисканню вбудованої кнопки **BOOT**.

Інструменти: Visual Studio Code з розширенням ESP-IDF , або середовище Espressif IDE. Плати Lolin C3 SuperMini з мікроконтроллером ESP32 C3.

Звіт: у зошиті. Орієнтовний зразок звіту у Teams, файл **Report_Example.pdf**

Вказівки до роботи

1. У звичайну програму «Blink» (Лабораторна робота N1) додайте бібліотеки:

```
8  #include <stdio.h>
9  #include "esp_wifi.h"
10 #include "esp_event.h"
11 #include "freertos/FreeRTOS.h"
12 #include "freertos/task.h"
13 #include "driver/gpio.h"
14 #include "sdkconfig.h"
15 #include <portmacro.h>
16 #include "esp_ota_ops.h"
17 #include "esp_http_client.h"
18 #include "esp_https_ota.h"
19 #include "esp_log.h"
20 #include "nvs_flash.h"
```

2. Додайте під'єднання до **WiFi** (із логуванням успішності).
3. Додайте код (функцію) для **OTA** оновлення:

```
56 void do_ota()
57 {
58     esp_http_client_config_t cfg = {
59         .url = OTA_URL,
60         .skip_cert_common_name_check = true,
61         .timeout_ms = 20000,
62     };
63
64     esp_https_ota_config_t https_ota_config = {
65         .http_config = &cfg,
66     };
67
68     esp_err_t r = esp_https_ota(&https_ota_config);
69     if (r == ESP_OK) {
70         esp_restart();
71     } else {
72         ESP_LOGE(TAG, "Update failed: %s", esp_err_to_name(r));
73     }
74 }
```

4. У кореневій папці проекту створіть файл для **partitions.csv** наступного змісту:

#	Name,	Type,	SubType,	Offset,	Size,	Flags
nvs,	data,	nvs,		0x9000,	0x4000,	
otadata,	data,	ota,		0xd000,	0x2000,	
app0,	app,	ota_0,		0x10000,	0x150000,	
app1,	app,	ota_1,		0x160000,	0x90000,	

5. Запустіть **idf.py menuconfig** та налаштуйте **Partition Table**:

```
( ) Single factory app, no OTA  
( ) Single factory app (large), no OTA  
( ) Factory app, two OTA definitions  
( ) Two large size OTA partitions  
(X) Custom partition table CSV
```

6. Там же, налаштуйте **Component Config** -> **ESP HTTPS OTA** -> **Allow HTTP**:

```
[ ] Provide decryption callback  
[*] Allow HTTP for OTA (WARNING: ONLY FOR TESTING PURPOSE, READ HELP)
```

7. На початку програми, перед приєднанням до **WiFi**, додайте виклик:

```
ESP_ERROR_CHECK(nvs_flash_init());
```

8. Ініціалізуйте **OTA_URL** значенням, заданим викладачем.
9. Пропишіть назву мережі **WiFi** та пароль.
10. Налаштуйте реакцію на натискання кнопки **BOOT** і викличте там **do_ota()** (один раз!)
11. Зробіть **idf.py fullclean** і скомпілюйте проєкт.
12. Знайдіть у директорії **build** файл ***.bin** та переконайтесь, що він не більший за **1.3Мб**.
13. Запрограмуйте плату.
14. Запустіть **Serial monitor**.
15. Натисніть **BOOT** і спостерігайте за повідомленнями у терміналі.
16. Повторюйте попередні кроки доти, доки усе не запрацює належним чином.
17. Якщо вивід нагадує відлагоджувальну інформацію з дампами пам'яті, регістрами, тощо – значить конфігурація була задана некоректно і чіп циклічно виходить на перезавантаження. Зупинивши монітор (кілька натискань **Ctrl+C**), можна побачити там змістовне повідомлення про помилку.

Контрольні запитання

1. Що таке OTA і для чого воно потрібно?